

Software Requirements and Design Document

For

Group Hoos' Fishin'

Version 1.0

Authors:

Juan Pablo Pastrana II

Tyler Dussault

Bridget Gizzi

Darius Davis

Dhruv Patel

1. Overview (5 points)

Hoo's Fishin' is a 2D fishing game where the player controls an owl named Hoo. The main gameplay loop centers on finding and catching fish, storing catches, selling fish to earn money, and improving fishing equipment. Different fish have their own values, rarity, and catch difficulty, while fishing rods can affect what fish can be caught and how quickly the fishing progress increases. The current system provides the foundation for this progression loop through player movement, fish spawning, a fishing minigame, inventory and hotbar management, a shop, a wallet, and fishing rod upgrades, with these systems working together as one playable experience.

2. Functional Requirements (10 points)

The following functional requirements describe the features currently implemented or supported in the increment 1 version of Hoo's Fishin' based on the current project code. These requirements reflect the functionality available at this stage of development and may be expanded or changed in later increments.

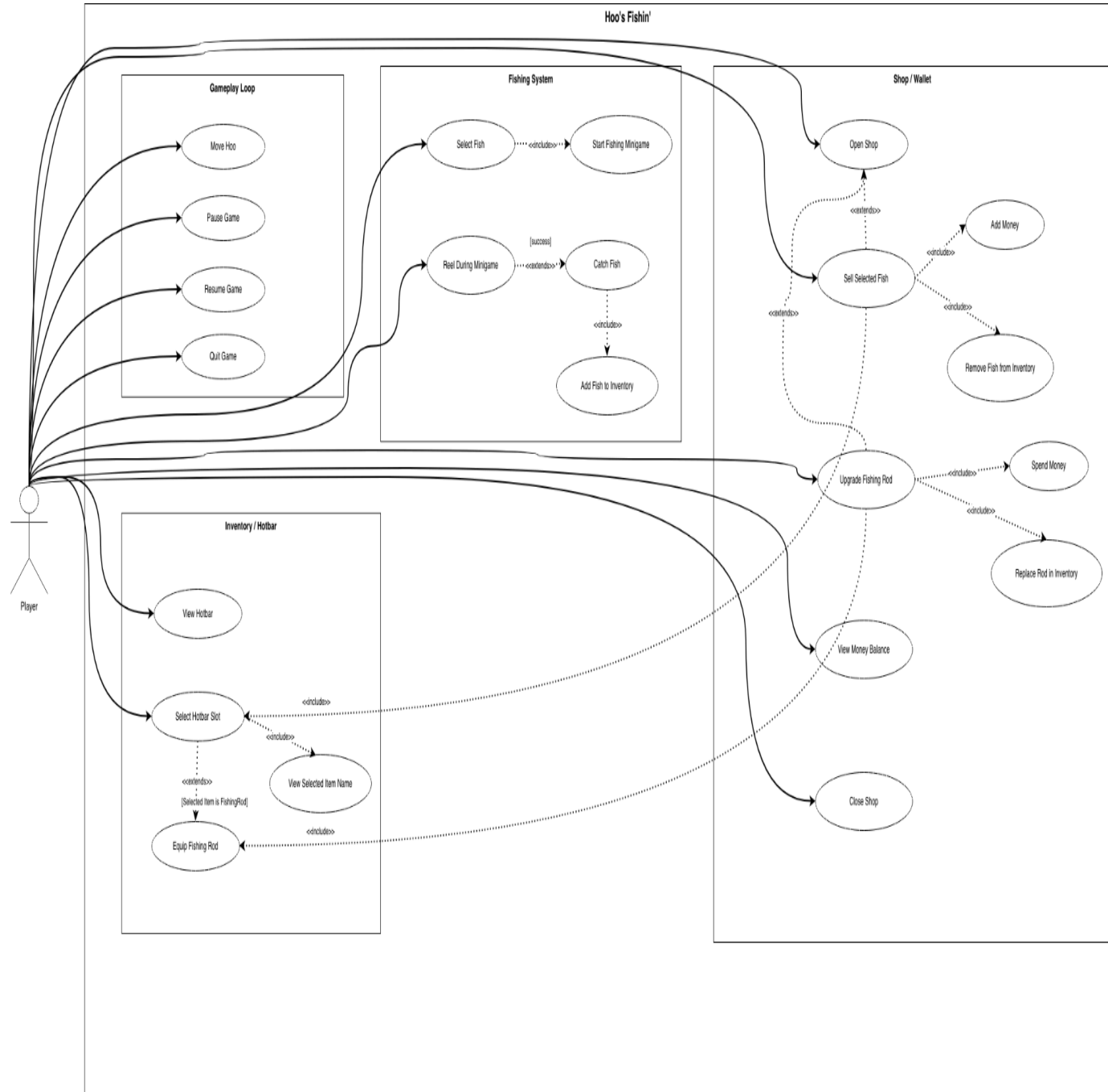
1. High - The system lets the player move Hoo around the game world
2. High - The system lets the player move Hoo around the game world.
3. High - The system lets the player interact with fish in the game world.
4. High - The system starts the fishing minigame when the player selects a fish.
5. High - The system determines whether a fishing attempt is successful or unsuccessful.
6. High - The system adds a successfully caught fish to the player's inventory.
7. High - The system stores multiple copies of the same fish in one inventory slot when stacking is available.
8. High - The system displays the player's inventory items in the hotbar.
9. Medium - The system displays the quantity of stacked items in the hotbar.
10. Medium - The system lets the player switch between hotbar slots.
11. Medium - The system displays the name of the currently selected item.
12. High - The system lets the player sell caught fish.
13. High - The system adds the value of a sold fish to the player's total money.
14. Medium - The system displays the player's current amount of money.
15. Medium - The system supports different fish with different names, values, and fishing difficulty.
16. Medium - The system spawns fish into the game world.
17. Medium - The system allows fish to move within the game world.
18. Medium - The system supports fishing rods with different attributes.
19. Medium - The equipped fishing rod affects the fishing minigame.
20. Medium - The system supports purchasing or upgrading fishing equipment.
21. High - The system prevents purchases when the player does not have enough money.
22. Low - The system lets the player pause and resume the game.
23. Low - The pause menu lets the player quit the game.

3. Non-functional Requirements (10 points)

1. Look and feel - The system is visually consistent such as being 2D pixel art that has no blur and wrong scaling.
2. Maintainability - The system stores data as Godot resources so that new fish and fishing rods can be added without changing the code.
3. Reliability
 - a. The system does not crash.
 - b. The system does not allow users to achieve negative balances.
 - c. The system does not allow inventory counts to be inaccurate.

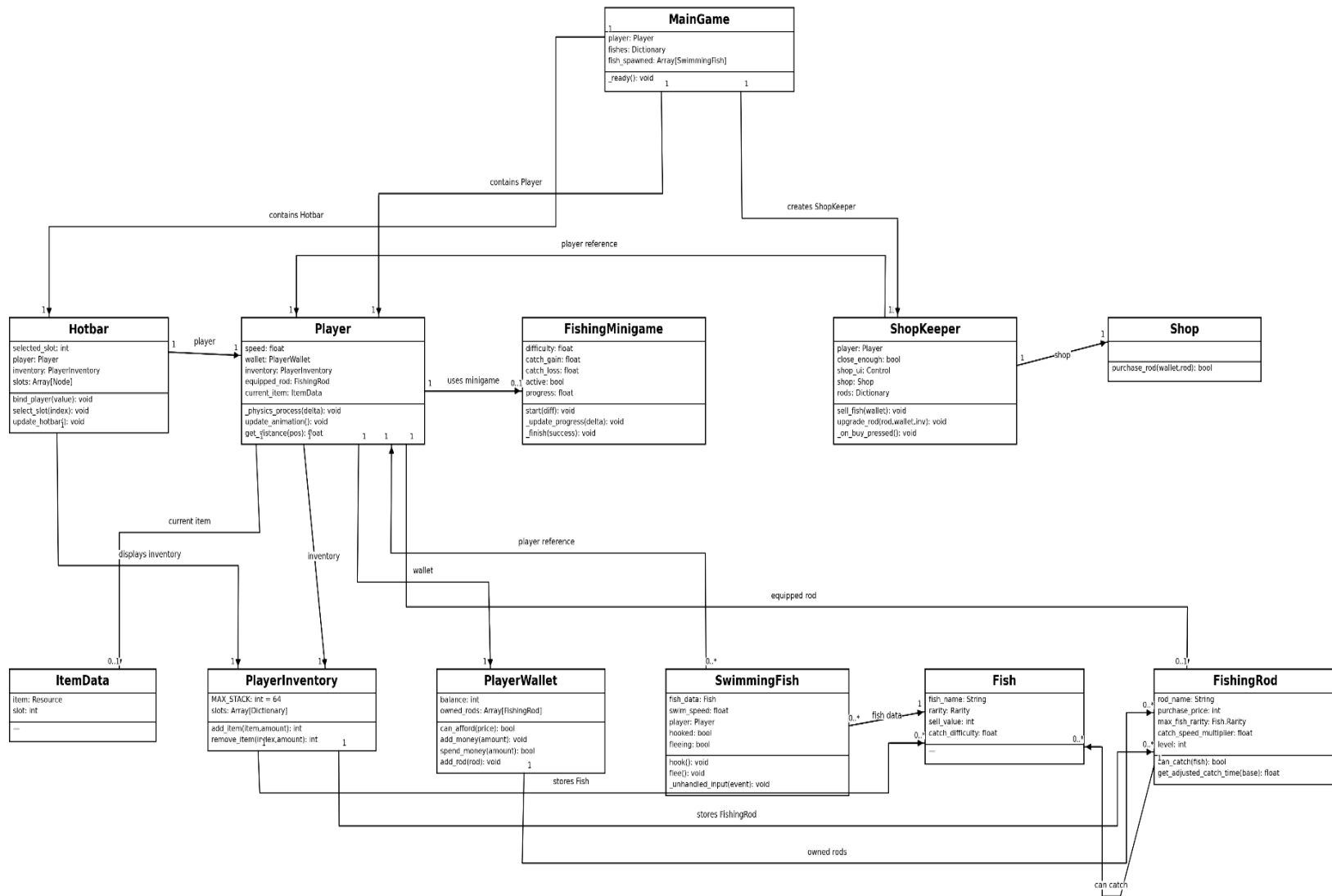
- The system allows new users to pick up the game and learn how to play without having to be told externally.
- The system allows controls to be consistent and information is displayed such as balance and items.

- #### 4. Use Case Diagram (10 points)

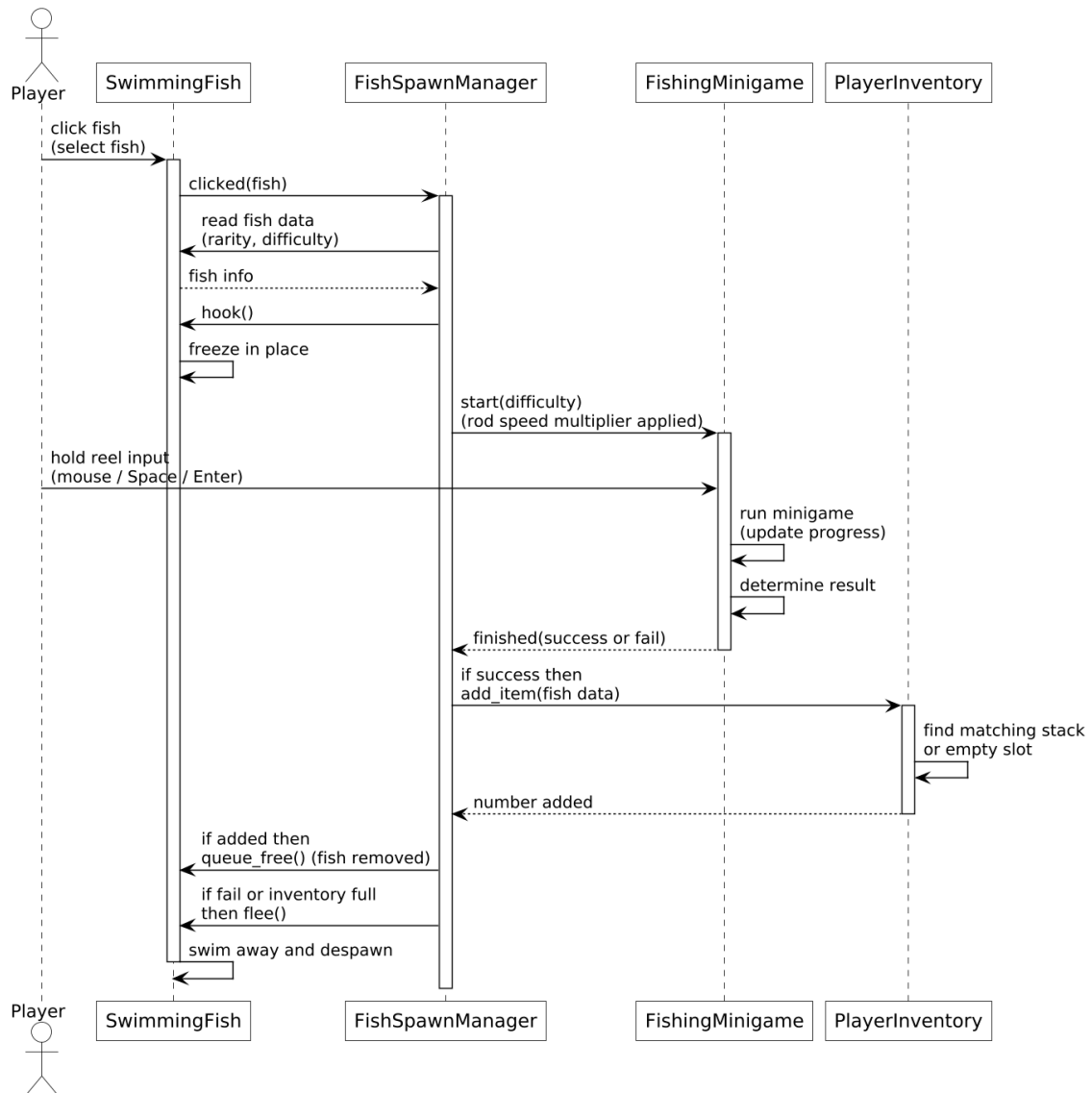
[illegible]

5. Class Diagram and/or Sequence Diagrams (15 points)

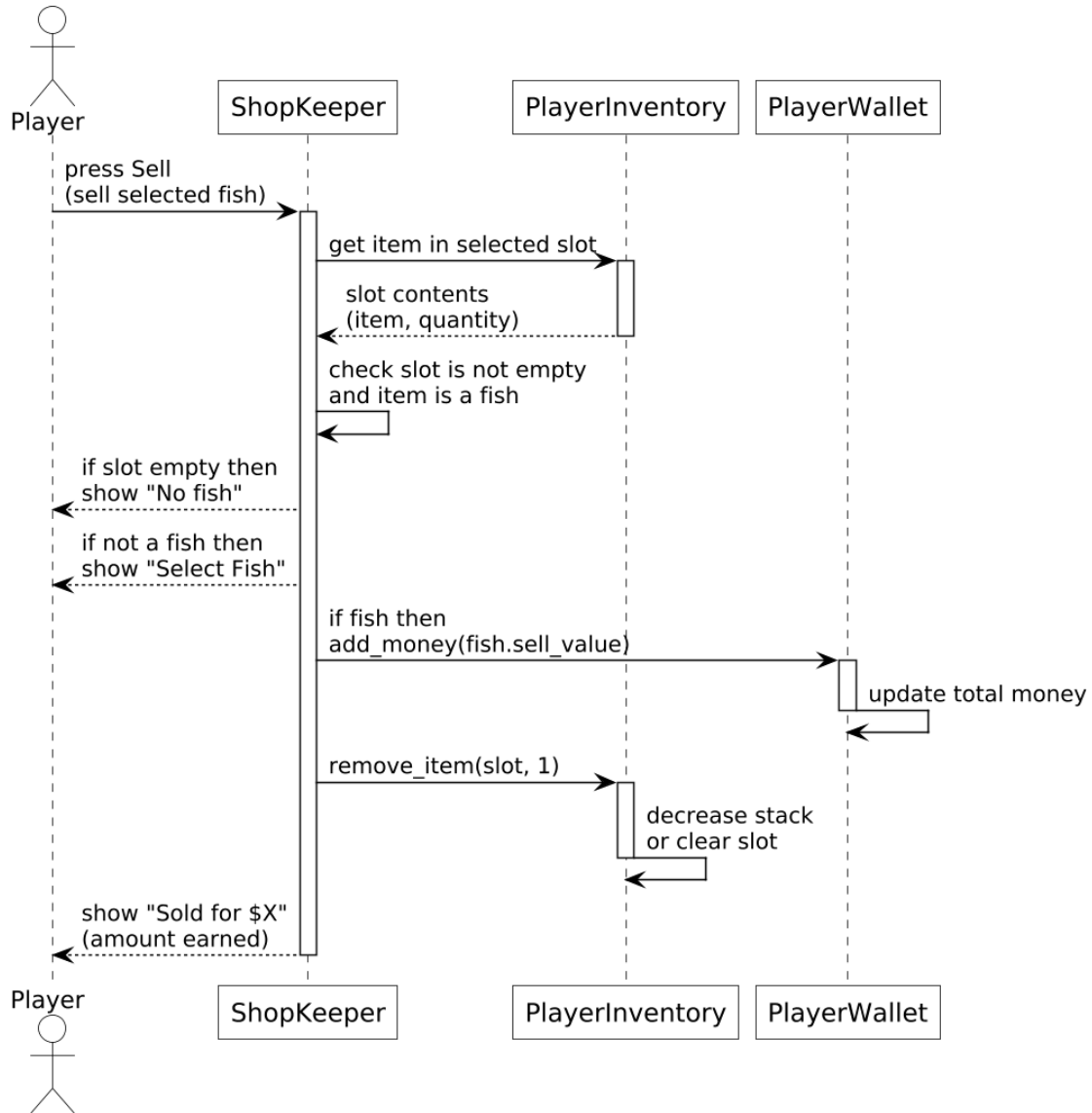
Class diagram



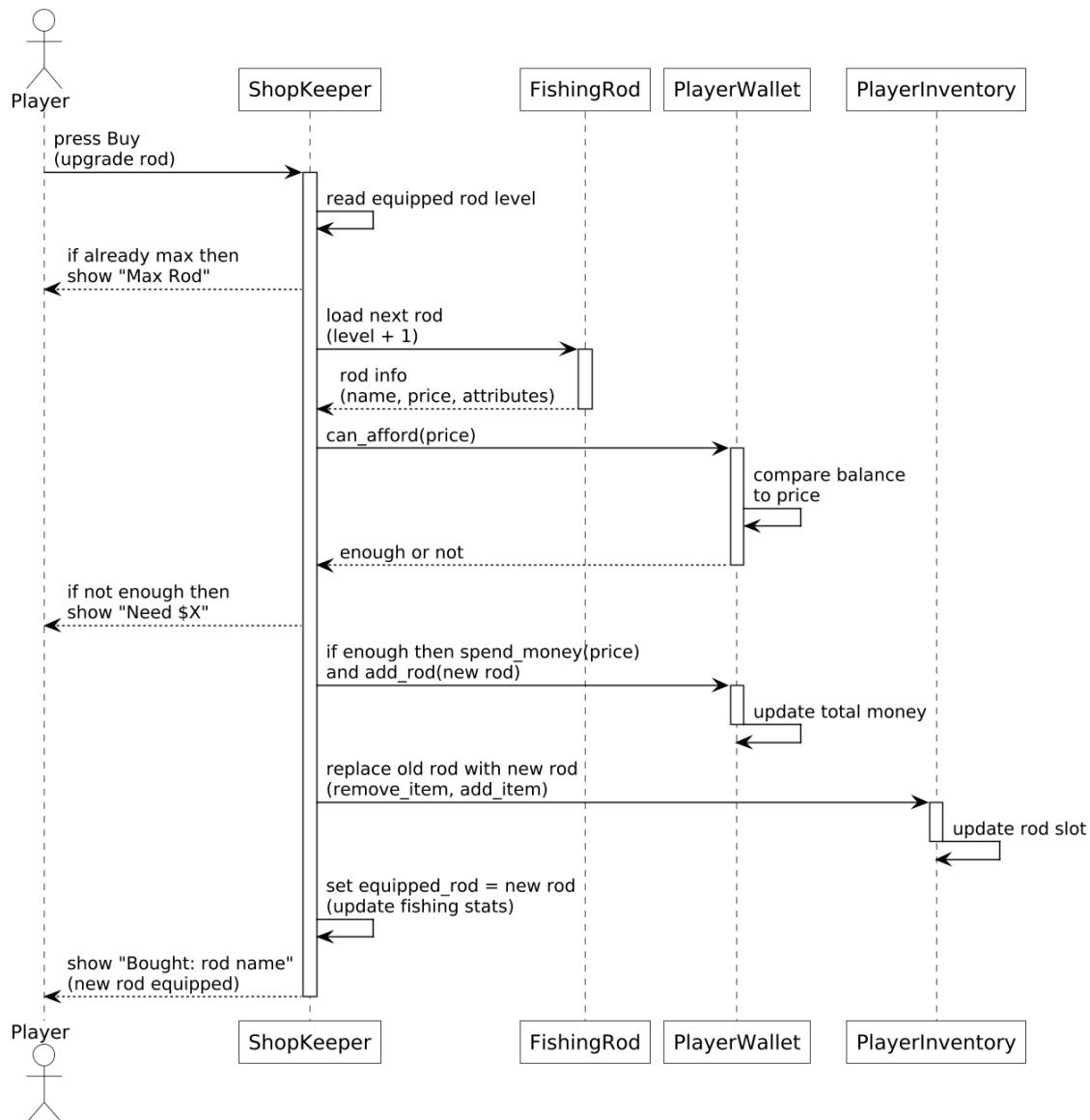
Catch fish sequence diagram



Sell fish sequence diagram



Upgrade + buy fishing rod sequence diagram



6. Operating Environment (5 points)

Since Hoo's Fishin' has not yet been released, this section describes the environment it is being developed in and is intended to run on. The game runs on Godot Engine 4.7 using Godot's Compatibility (OpenGL-based) renderer, chosen for its low hardware requirements and support for older or integrated graphics cards. It renders at 384×216 and scales to fill the window, standard for pixel-art games. Our team has developed and tested primarily on Windows, though Godot 4 is cross-platform and also supports macOS and Linux. Hardware requirements are minimal: any computer capable of running the Godot 4 editor with OpenGL 3.3 support, plus a keyboard and mouse (arrow keys/WASD to move, number keys for the hotbar, mouse to interact

with fish). The game currently runs through the Godot editor, as no standalone export build exists yet. In development, it also depends on Git/GitHub for version control, though this will not be required by end users. A future increment will include exporting to a standalone executable, after which no additional software will be needed to run the game.

7. Assumptions and Dependencies (5 points)

We are assuming that Godot and the open source tools used in development will remain available and continue to support the versions we are using throughout the project. If these tools changed significantly or became unavailable, it could affect our development process. The project depends on Godot Engine 4.7 as its core third party component. Since Godot is free and open source there are no costs or restrictions to account for. The project also depends on GitHub for version control, and we are assuming its free tier will continue to be sufficient for our needs. Because the game does not rely on internet connectivity or an external database, it is not dependent on any third party servers or APIs, which reduces external risk. We are also assuming that future increments, such as exporting a standalone executable, will not require additional paid tools or licenses. If any of these assumptions are incorrect or change, the requirements described in this document may need to be revisited.